**OOP and Design Patterns (CSCI 375)**
**Student Showcase (Final Project) Rubric**

1. Project Title: File Sorter

**Instructions:**

1. There are 10 technical requirements (worth 100 points) to grade the project and the team presentation.
2. For each requirement, use 0 - 10 scale in the Score column (0-6: Fail, 7: C (Average), 8: B (Above Average), 9: A (Good), (10: Excellent)
3. Use the *Notes* section to jot down any observations that may help in grading and justification.

| **Team and Technical Project Requirement** | **Score** |
|---|---|
| 1. Use **4+1 Views** to explain the software design to the audience.<br>Notes: Not all views displayed, but there is enough there to understand how the program works. | 10/10 |
| 2. **Use of 3 Design Patterns** -- presentation clearly stated and briefly explained design patterns use. Common design patterns are Iterator, Decorator, Observer, Strategy, Command, State, Singleton, Adapter, Façade, Flyweight, Abstract Factory, Composite, Template, MVC, etc.<br>Notes: GUI – Singleton. Observer/call-backs, helper functions | 9/10 |
| 3. **Use of Fundamental OOD Concepts-** e.g., Inheritance, Abstraction, Getters and Setters, Overloading, Attributes and Methods, etc.<br>Notes: Used getters/setters to access data fields | 10/10 |
| 4. **Software management** – good usage of management, communication and tracking tools e.g., Gant chart, Kanban board, GitHub Project, Clickup, Discord, Slack, etc.<br>Notes: Github used to manage codebase, in-person and text communication | 10/10 |
| 5. **Documentation** – clear, easy to follow documentation, UML diagrams are complete, and notations are correct; explanation of objects interaction is clear and complete.<br>Notes: HTML docs and UML diagrams included. UML diagrams are simple but effective | 9/10 |

| | |
|---|---|
| 6. **Testing and CI-CD pipeline – automatically generates test data** using hypothesis, usage of mocking/patching, provides code coverage and Python type check (mypy) reports, CI-CD, Docker, etc.<br><br>Notes: 100% test coverage | 10/10 |
| 7. **Clean code and Modularity** – project uses appropriate software and system design; the code follows best practices (pep8), is well organized and refactored into packages, modules, classes, etc.<br>Notes: passes flake8 and mypy tests. Makefile manages dependencies and compilation. | 10/10 |
| 8. **Project requirements and execution** -- clearly stated functional and technical requirements, project adequately challenging for sophomore-junior students; project demo was clear and concise, etc.<br>Notes: This was my first time developing a full stack application, and I learned a lot about front-end and full-stack development | 10/10 |
| 9. **Team presentation** -- all members participated in presentation, used the visual and oral presentation techniques and tools to engage audience, etc.<br>Notes: Equal separation of the presentation | 9/10 |
| 10. **Individual Contributions** – briefly explain how and what each member contributed to the successful execution of the project.<br>Notes: Alex developed the GUI and EventHandler with a little help from Brian, and I helped Brian develop the classes and tags system. | 10/10 |
| 11. **BONUS: Above and beyond** – Team went beyond the above list e.g., great User Interface, use of Database, security, real-world application, real-world client delight and interaction, etc.<br>Notes: Brain and Alex deserve most of the credit here. I was mostly just support for this project since I've taken the class before. | 5/10 |

| | |
|---|---|
| **Total Score**<br><br>Note: Max score can be 110 due to 10 BONUS points. | 102/100 |